1 部署Flask网站
1.1 介绍
1.2 获取镜像+创建代码
1.2.1 获取基础镜像
1.2.2 dockerfile
1.2.3 创建app.py
1.3 构建镜像+运行+删除
1.3.1 构建镜像
1.3.2 基于基础镜像创建容器+运行
1.3.3 删除
2 Docker
2.1 dockerfile指令-创建镜像运行
2.1.1 From
2.1.2 MAINRTAINER(has been deprecated)
2.1.3 ENV
2.1.4 COPY|ADD
2.1.5 WORKDIR
2.2 dockerfile指令-容器运行
2.2.1 CMD
2.2.2 ENTRYPOINT
2.2.3 注意
2.2.4 扩展
2.3 docker 构建（重要）
2.4 容器
2.4.1 创建容器
2.4.2 容器常见操作
2.4.3 容器销毁
2.5 常见操作
3 实践：gradio+openai
3.1 介绍
3.2 dockerfile
3.3 python代码
3.4 构建镜像+运行
3.5 导入导出和发布
3.5.1 本地导出
3.5.2 本地导入
3.5.3 发布
4 命令汇总
4.1 镜像构建
4.2 镜像命令
4.3 容器命令
4.4 查看日志
4.5 docker推送

1 部署Flask网站

1.1 介绍

- 需求：基于docker创建在ubuntu18.04系统上运行的Flask网站
- 流程：
    1. 在centos7.9上安装docker
    2. 获取基础镜像 ubuntu:18.04
    3. 在基础镜像基础上构建自定义镜【ubuntu18.04+python+代码】
    4. 基于镜像创建容器+运行

1.2 获取镜像+创建代码

1.2.1 获取基础镜像

- 基础镜像为ubuntu:18.04

> docker pull ubuntu:18.04

1.2.2 dockerfile

- Dockerfile

```
# Dockerfile

# Base images 基础镜像
FROM ubuntu:18.04

# MAINTAINER 维护者信息
MAINTAINER qinchihongye@gmail.com

# RUN 执行以下命令
RUN apt update
RUN apt install python3 python3-pip -y
RUN pip3 install flask
RUN mkdir -p /data/www/

# 拷贝文件至工作目录
COPY app.py /data/www/

# 工作目录
WORKDIR /data/www/

# EXPOSE 映射端口
```

```
EXPOSE 80

# 容器启动时执行命令
CMD ["python3","app.py"]
```

1.2.3 创建app.py

- app.py

```
from flask import Flask

app = Flask(__name__)

@app.route("/index")
def index():
    return "这里是docker+flask的示例"

if __name__ == "__main__":
    app.run(host='0.0.0.0',port=8000)
```

1.3 构建镜像+运行+删除

1.3.1 构建镜像

- 基于dockerfile+app.py

> docker build -t mengzhichao/web1:1.0 . -f Dockerfile

下面给出参数的解释

1. `docker build`：这是Docker命令行工具中用于构建镜像的命令。
2. `-t`：这是 `--tag` 的缩写，用于给新创建的镜像指定一个标签（tag）。标签通常用于区分不同的镜像版本。
3. `mengzhichao/web1:1.0`：这是指定的镜像名称和标签。在这个例子中，`mengzhichao` 是Docker Hub上的用户名，`web1` 是镜像的名称，而 `1.0` 是这个镜像的版本标签。
4. `.`：这是构建上下文的路径。`.` 表示当前目录是构建上下文，Dockerfile和其他需要的文件都应该位于当前目录下。
5. `-f Dockerfile`：这个选项用于指定构建过程中使用的 Dockerfile 的名称。默认情况下，Docker 会寻找名为 `Dockerfile` 的文件，但如果你使用了不同的文件名，就需要通过 `-f` 选项来指定。在这个例子中，指定了 `Dockerfile` 作为构建指令集的文件，即使文件名是默认的 `Dockerfile`，也可以明确指出。

1.3.2 基于基础镜像创建容器+运行

- 代码

> docker run -d -p 80:8000 --name mengzhichao mengzhichao/web1:1.0

各部分的含义：

1. `docker`： 这是Docker命令行工具的命令。
2. `-d`： 这是“detach”的缩写，表示以守护进程模式运行容器，即在后台运行。
3. `-p 80:8000`： 这是端口映射的设置。`-p` 后面跟的是宿主机（主机）端口和容器端口的映射关系。这里表示将容器内部的8000端口映射到宿主机的80端口。这样，当访问宿主机的80端口时，实际上是访问容器内部的8000端口。
4. `--name mengzhichao`： 这指定了容器的名称。这里容器的名称被设置为 `mengzhichao` 。
5. `mengzhichao/web1:1.0`： 这是要运行的Docker镜像的名称和标签。格式通常为 `用户名/仓库名:标签` 。在这个例子中， `mengzhichao` 是Docker Hub上的用户名， `web1` 是仓库名， `1.0` 是标签，表示使用的是这个仓库中标签为1.0的镜像。

综上所述，这个命令的作用是：在后台启动一个名为 `mengzhichao` 的Docker容器，使用 `mengzhichao/web1:1.0` 镜像，并将容器内部的8000端口映射到宿主机的80端口。

1.3.3 删除

- 查看镜像

> docker images

- 查看正在运行的容器

> docker ps

- 查看所有容器（已创建未运行的，停止的，错误的）

> docker ps -a

- 删除容器

> docker rm 容器id

- 删除镜像

> docker rmi 镜像名称

---

2 Docker

2.1 dockerfile指令-创建镜像运行

2.1.1 From

- FROM 指定基础镜像，必须为第一个命令

```
格式:
  FROM <image>
  FROM <image>:<tag>
示例:
  FROM mysq7:5.7
注意:
  1. tag是可选的,如果不使用tag时,会使用latest版本的基础镜像
  2. 基础镜像可以是官方的镜像，也可以是自定义的镜像
```

2.1.2 MAINRTAINER(has been deprecated)

- MAINTAINER 镜像维护者的信息

```
格式:
  MAINTAINER <name>
示例:
  MAINTAINER mengzhichao
  MAINTAINER qinchihongye@gmail.com
  MAINTAINER mengzhichao qinchihongye@gmail.com
```

2.1.3 ENV

- ENV 相当设置系统的环境变量

```
格式:
  ENV <key> <value>
  ENV <key>=<value>
示例:
  ENV myName John
  ENV mycat=fluffy
```

2.1.4 COPY|ADD

- COPY|ADD添加本地文件到镜像中

```
格式：
  COPY <src>...<dest>
示例：
  ADD hom* /mydir/ # 添加所有以"hom"开头的文件到/mydir/
  ADD test relativeDir/ # 添加 "test" 到 $WORKDIR/relativeDir/
  ADD test /absoluteDir/ # 添加 "test" 到 /absoluteDir/
注意：
  针对压缩包使用ADD的话,会自动解压。如果想保留压缩包用COPY，否则用ADD
```

2.1.5 WORKDIR

- WORKDIR 工作目录

```
格式：
  WORKDIR /path/to/workdir
示例：
  WORKDIR /a （这时工作目录为/a）
注意：
  通过WORKDIR设置工作目录后，Dockerfile中其后的命令RUN、CMD、ENTRYPOINT、ADD、COPY等命令都会在该目录下执行
```

2.2 dockerfile指令-容器运行

2.2.1 CMD

- CMD 构建容器后调用，也就是在容器启动时才进行调用

```
格式：
    CMD ["executab7e","param1","param2"] （执行可执行文件,优先）
    CMD ["param1","param2"] （设置了ENTRYPOINT,则直接调用ENTRYPOINT添加参数）
    CMD command param1 param2 （执行shell内部命令）
示例：
    CMD ["/usr/bin/wc","--help"]
    CMD ping www.baidu.com
注意：
  CMD不同于RUN,CMD用于指定在容器启动时所要执行的命令,而RUN用于指定镜像构建时所要执行的命令。
```

2.2.2 ENTRYPOINT

- ENTRYPOINT 设置容器初始化命令，使其可执行化

```
格式:
  ENTRYPOINT ["executab7e", "param1", "param2"] (可执行文件, 优先)
  ENTRYPOINT command param1 param2 (she11 内部命令)
示例:
  ENTRYPOINT ["/usr/bin/wc","--he1p"]
注意:
  ENTRYPOINT与CMD非常类似, 不同的是通过docker run执行的命令不会覆盖ENTRYPOINT, 而docker run命令中指定的任何参数, 都会被当做参数再次传递给ENTRYPOINT。Dockerfile中只允许有一个ENTRYPOINT命令, 多指定时会覆盖前面的设置, 而只执行最后的ENTRYPOINT指令
```

2.2.3 注意

- CMD和ENTRYPOINT只有最后一条命令生效
- 因为 CMD为容器运行时执行的指令，所以在容器运行时时可以被覆盖的

  如下

  ```
  # 容器启动时执行命令
  CMD ["python3","app.py"]
  ```

  如果在运行时使用以下命令

  > docker run -d -p 80:8000 --name mengzhichao mengzhichao/web1:1.0

  则容器终端会运行

  > python3 app.py

  但是如果运行镜像时使用如下命令

  > docker run -d -p 80:8000 --name mengzhichao mengzhichao/web1:1.0 python3 test.py

  那么容器终端会运行

  > python3 test.py

当然一般我们也不会去加

- 但是ENTRYPOINT 不会被覆盖，而是执行

  > python3 app.py python3 test.py

  即 `python3 test.py` 会被当成参数传入

2.2.4 扩展

- 因为CMD和ENTRYPOINT只有最后一条命令生效，如果想有多条命令，可以先将命令写如sh

文件，再使用ENTRYPOINT调用shell文件实现

> ENTRYPOINT ['/bin/sh','run.sh']

当然run.sh得提前copy到workdir

2.3 docker 构建（重要）

- docker在构建镜像时会有缓存,这样二次构建的时候会很快，比如我们在已经有 `mengzhichao/web1:1.0` 的基础上构建 `mengzhichao/web1:1.1`

  如果运行

> docker build -t mengzhichao/web1:1.1 . -f Dockerfile

```
C:\Users\孟智超\Desktop\mac_backups\学习\docker\1_部署FLask网站>docker build -t mengzhichao/web1:1.1 . -f Dockerfile
[+] Building 0.4s (12/12) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 490B
 => [internal] load metadata for docker.io/library/ubuntu:18.04
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [1/7] FROM docker.io/library/ubuntu:18.04
 => [internal] load build context
 => => transferring context: 28B
 => CACHED [2/7] RUN apt update
 => CACHED [3/7] RUN apt install python3 python3-pip -y
 => CACHED [4/7] RUN pip3 install flask
 => CACHED [5/7] RUN mkdir -p /data/www/
 => CACHED [6/7] COPY app.py /data/www/
 => CACHED [7/7] WORKDIR /data/www/
 => exporting to image
 => => exporting layers
 => => writing image sha256:4194ba1ce11ea9225ed8929cca0df4a24a732715ef9920b33e281e2ffe08a5fe
 => => naming to docker.io/mengzhichao/web1:1.1
```

这样会利用到上次构建时缓存的内容，所以应该

> docker build -t mengzhichao/web1:1.1 . -f Dockerfile --no-cache

```
C:\Users\孟智超\Desktop\mac_backups\学习\docker\1_部署FLask网站>docker build -t mengzhichao/web1:1.1 . -f Dockerfile --no-cache
[+] Building 4.2s (5/11)
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 490B
 => [internal] load metadata for docker.io/library/ubuntu:18.04
 => [internal] load .dockerignore
 => => transferring context: 2B
 => CACHED [1/7] FROM docker.io/library/ubuntu:18.04
 => [internal] load build context
 => => transferring context: 28B
 => [2/7] RUN apt update
 => => # Get:3 http://security.ubuntu.com/ubuntu bionic-security/multiverse amd64 Packages [23.8 kB]
 => => # Get:4 http://archive.ubuntu.com/ubuntu bionic-updates InRelease [102 kB]
 => => # Get:5 http://security.ubuntu.com/ubuntu bionic-security/main amd64 Packages [3373 kB]
 => => # Get:6 http://archive.ubuntu.com/ubuntu bionic-backports InRelease [102 kB]
 => => # Get:7 http://archive.ubuntu.com/ubuntu bionic/main amd64 Packages [1344 kB]
 => => # Get:8 http://archive.ubuntu.com/ubuntu bionic/universe amd64 Packages [11.3 MB]
```

这样里面的则会重新运行如apt install,git clone等命令，以防有版本升级或者代码改动

所以如果有类似于git clone这类的，记得加 no-cache

2.4 容器

2.4.1 创建容器

- 注意事项

1. 容器中必须有前台进程, 创建立即销毁。比如打开nginx

> systemctl start nginx

如果使用以上命令，则会在后台运行，没有前台进程卡住

必须使用以下命令，才有后台进程卡住

> nginx -c /etc/nginx/myblog.conf -g 'daemon off;'

2. 宿主机centos7.9运行创建容器是否需要前台?

- 宿主机后台进程 + 容器内部前台进程

> docker run -d -p 80:8000 mengzhichao/web1:1.0

```
C:\Users\孟智超\Desktop\mac_backups\学习\docker\1_部署FLask网站>docker ps
CONTAINER ID   IMAGE           COMMAND                  CREATED       STATUS        PORTS                      NAMES
c08a61b30341   ollama/ollama   "/bin/ollama serve"      5 days ago    Up 3 hours    0.0.0.0:11434->11434/tcp   ollama

C:\Users\孟智超\Desktop\mac_backups\学习\docker\1_部署FLask网站>docker run -d -p 80:8000 mengzhichao/web1:1.0
21fbce6c52372f5dece54282180731760728cfdbc0e6e7651d77967b18a4c962

C:\Users\孟智超\Desktop\mac_backups\学习\docker\1_部署FLask网站>docker ps
CONTAINER ID   IMAGE                 COMMAND                  CREATED         STATUS         PORTS                          NAMES
21fbce6c5237   mengzhichao/web1:1.0  "python3 app.py"         6 seconds ago   Up 4 seconds   80/tcp, 0.0.0.0:80->8000/tcp   vigilant_buck
c08a61b30341   ollama/ollama         "/bin/ollama serve"      5 days ago      Up 3 hours     0.0.0.0:11434->11434/tcp       ollama
```

- 仅容器内部前台进程(没有-d,则前台运行)

> docker run -p 80:8000 mengzhichao/web1:1.0

```
C:\Users\孟智超\Desktop\mac_backups\学习\docker\1_部署FLask网站>docker run -p 80:8000 mengzhichao/web1:1.0
 * Serving Flask app 'app' (lazy loading)
 * Environment: production
   WARNING: This is a development server. Do not use it in a production deployment.
   Use a production WSGI server instead.
 * Debug mode: off
 * Running on all addresses.
   WARNING: This is a development server. Do not use it in a production deployment.
 * Running on http://172.17.0.3:8000/ (Press CTRL+C to quit)
```

- 仅宿主机后台进程，没有前台进程卡住，创建即销毁

以直接启动ubuntu为例子

> docker run -d -p 80:8000 ubuntu:18.04

```
C:\Users\孟智超\Desktop\mac_backups\学习\docker\1_部署FLask网站>docker ps
CONTAINER ID   IMAGE           COMMAND                  CREATED       STATUS        PORTS                     NAMES
c08a61b30341   ollama/ollama   "/bin/ollama serve"      5 days ago    Up 3 hours    0.0.0.0:11434->11434/tcp  ollama

C:\Users\孟智超\Desktop\mac_backups\学习\docker\1_部署FLask网站>docker run -d -p 80:8000 ubuntu:18.04
7489721df91890991f200d67a1cd4fb2387fd775c38442b445516545d3f9cba9

C:\Users\孟智超\Desktop\mac_backups\学习\docker\1_部署FLask网站>docker ps
CONTAINER ID   IMAGE           COMMAND                  CREATED       STATUS        PORTS                     NAMES
c08a61b30341   ollama/ollama   "/bin/ollama serve"      5 days ago    Up 3 hours    0.0.0.0:11434->11434/tcp  ollama

C:\Users\孟智超\Desktop\mac_backups\学习\docker\1_部署FLask网站>
```

2.4.2 容器常见操作

- 镜像含前台进程，则创建容器+启动

> docker run -d -p 80:8000 --name mengzhichao mengzhichao/web1:1.0

- 镜像不含前台进程，创建容器+启动

> docker run -t ubuntu:18.04

1. 直接进入系统

> docker run -ti ubuntu:18.04 bash
>
> 或
>
> docker run -ti ubuntu:18.04 sh

```
C:\Users\孟智超\Desktop\mac_backups\学习\docker\1_部署FLask网站>docker run -ti ubuntu:18.04 bash
root@ec1632f4330b:/#
root@ec1632f4330b:/# pwd
/
root@ec1632f4330b:/# ls
bin  boot  dev  etc  home  lib  lib64  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
root@ec1632f4330b:/#
```

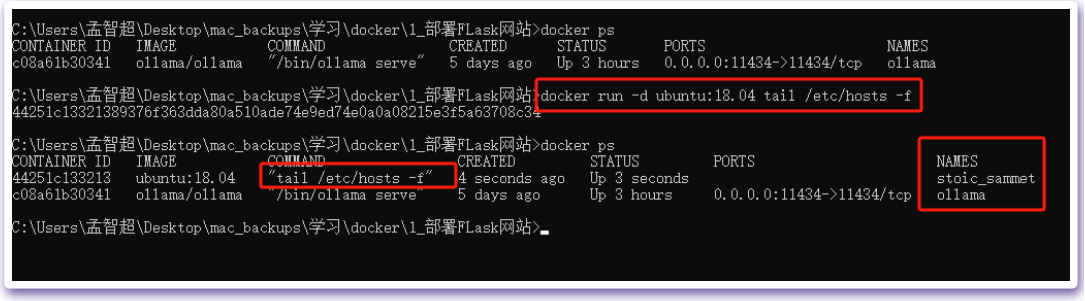
```
C:\Users\孟智超\Desktop\mac_backups\学习\docker\1_部署FLask网站>docker run -ti ubuntu:18.04 sh
#
# pwd
/
# ls
bin  boot  dev  etc  home  lib  lib64  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
# ^A
```

加 `--rm` 则推出后容器也自动销毁，否则容器需要手动销毁

```
docker run -ti --rm ubuntu:18.04 bash
```

2. 添加前台进程(认为，一般不做这个操作,只需知道可以这样做)

```
docker run -d ubuntu:18.04 tail /etc/hosts -f
```



2.4.3 容器销毁

- 先stop，再删除
- 只获取容器id

```
docker ps -aq
```

- 批量停止+删除(**谨慎使用**)

```
docker stop `docker ps -aq`
```

```
docker rm `docker ps -aq`
```

2.5 常见操作

- 镜像含前台进程, 创建容器+启动

```
docker run -d -p 80:8000 wupeiqi/web1:1.0
```

- 镜像不含前台进程, 今入操作

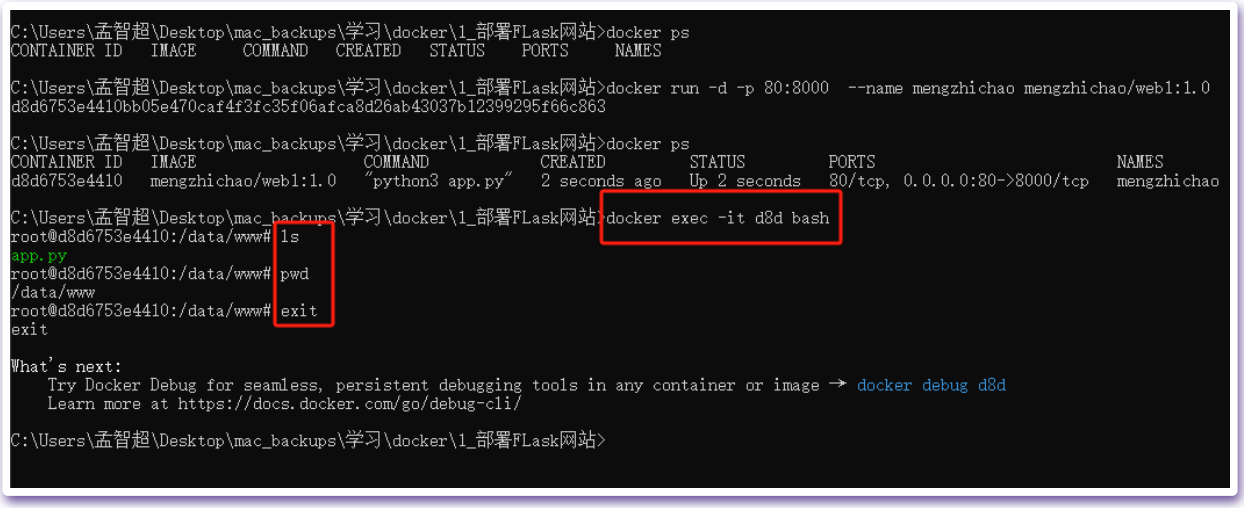
```
docker run -ti --rm ubuntu:18.04 bash
docker run -ti --rm ubuntu:18.04 sh
```

- 再次进入正在存活的容器

> docker exec -ti 容器ID bash

如

> docker ps
>
> docker run -d -p 80:8000 --name mengzhichao mengzhichao/web1:1.0
>
> docker exec -it 容器ID bash



3 实践：gradio+openai

3.1 介绍

- 需求：基于docker创建在python3.9上，使用gradio和openai调用openai大模型，传入baseurl,apikey和query获取答案
- 流程：
    1. 下载基础镜像 `python:3.9-slim`
    2. 构建自定义镜像 python3.9+代码
    3. 基于镜像创建容器+运行

3.2 dockerfile

- dockerfile

```
# Base images 基础镜像
FROM python:3.9-slim

# MAINTAINER 维护者信息
LABEL maintainer="qinchihongye@gmail.com"

# 更新软件包列表并安装 pip
RUN apt-get update && apt-get install -y --no-install-recommends \
    git \
    curl \
    && rm -rf /var/lib/apt/lists/*

# 安装 gradio, fastapi, uvicorn
RUN pip install --no-cache-dir gradio uvicorn openai

# 创建工作目录
RUN mkdir -p /data/

# 拷贝文件至工作目录
COPY gradio_llm.py /data/

# 工作目录
WORKDIR /data/

# EXPOSE 映射端口
EXPOSE 1234

# 容器启动时执行命令
CMD ["python3","gradio_llm.py"]
```

备注：暴露出1234端口

3.3 python代码

- gradio_llm.py

```python
import gradio as gr
import openai
from openai import OpenAI

def get_completion(query,api_key,base_url,model="gpt-3.5-turbo"):

    openai.api_key = api_key
    openai.base_url = base_url

    client = OpenAI(api_key=openai.api_key,base_url=openai.base_url)

    messages = [{"role": "user", "content": query}]
    response = client.chat.completions.create(
                                                model=model,
                                                messages=messages,
                                            )
    return response.choices[0].message.content

def process_query(base_url,api_key,query):
    response = get_completion(query=query,api_key=api_key,base_url=base_url)

    print(response)
    return response

with gr.Blocks(title='openai+gradio测试') as demo:

    with gr.Tab(label='图片'):
        with gr.Row():
            with gr.Column(scale=2):
                base_url_text = gr.Text(label='base_url',placeholder='https://api.xty.app/v1')
                api_key_text = gr.Text(label='api_key',placeholder='sk-RZH5BgtgdDazdwEZD4D50cC6Cd984a8dA838B482918d8e75')
                query_text = gr.Text(label='query for llm',placeholder='query')
                query_button = gr.Button(value='提问',variant='primary')

            with gr.Column(scale=5):
                output_text = gr.Markdown(label='回答结果')

    query_button.click(fn=process_query
```

```
                        ,inputs=[base_url_text,api_key_text,query_text]
                        ,outputs=[output_text]
                        )

demo.launch(server_port=1234,server_name='0.0.0.0')
```

备注：

1. `127.0.0.1` 仅能本机连接，而 `0.0.0.0` 既可以本机连接又可以别的机器连接
2. `server_name` 如果不指定，默认是 `127.0.0.1`

3.4 构建镜像+运行

- 构建镜像

> docker build -t mengzhichao/gradio_test:1.0 . -f Dockerfile

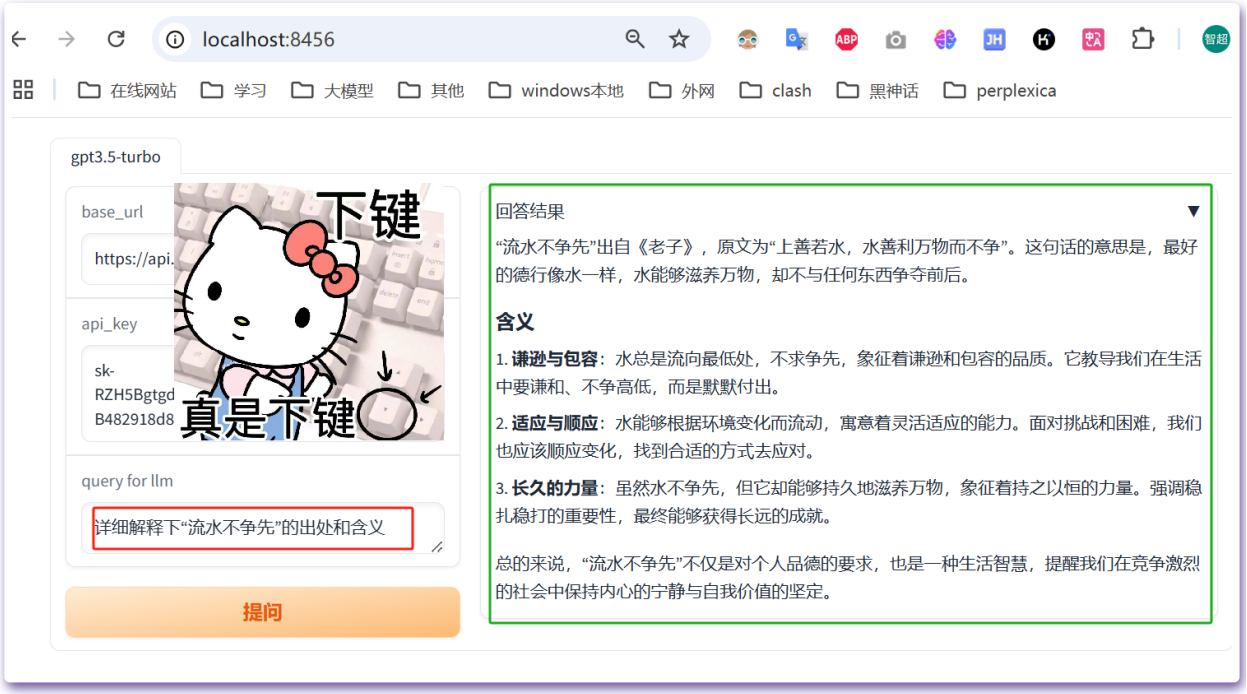
- 运行

> docker run -d -p 8456:1234 --name gradio_test mengzhichao/gradio_test:1.0

- 连接

> http://127.0.0.1:8456

- 页面



3.5 导入导出和发布

- 在使用dockerfile构建好镜像后，镜像就暂时存在本地镜像库了

3.5.1 本地导出

- **使用 `docker save` 命令：**

  这个命令可以将一个或多个镜像保存到一个 tar 归档文件中。

  ```
  docker save -o my_image.tar my_image:tag
  ```

  其中 `my_image:tag` 是你想要保存的镜像的名称和标签， `my_image.tar` 是保存的文件名。

  > docker save -o gradio_test.tar mengzhichao/gradio_test:1.0

3.5.2 本地导入

- **使用 `docker load` 命令：**

  这个命令用于从 tar 归档文件中加载一个镜像。

  ```
  docker load -i my_image.tar
  ```

  其中 `my_image.tar` 是你之前保存的镜像文件。

  > docker load -i gradio_test.tar

3.5.3 发布

- docker login

  > DockerId: qinchihongye
  > password:

  ```
  docker login -u qinchihongye
  ```

- push

  > docker push mengzhichao/gradio_test:1.0

- 后续使用

> docker pull mengzhichao/gradio_test:1.0

---

4 命令汇总

4.1 镜像构建

- 利用 dockerfile 构建镜像

```
docker build -t my-image-name:1.0 --no-cache -f Dockerfile.custom --build-arg USER_ID=1000 -m 2g --network=host .
```

1. `-t` 或 `--tag`: 指定镜像的名称和标签。
   my-image-name:1.0: 镜像的名称为 my-image-name，标签为 1.0。
2. `--no-cache`
   --no-cache: 构建镜像时不使用缓存，每次都从头开始构建。这在需要确保构建过程不受之前缓存影响时非常有用。
3. `-f` Dockerfile.custom
   -f 或 --file: 指定用于构建镜像的 Dockerfile 文件。
   Dockerfile.custom: 使用 Dockerfile.custom 文件进行构建，而不是默认的 Dockerfile。
4. `--build-arg` USER_ID=1000
   --build-arg: 向构建过程中传递构建参数。
   USER_ID=1000: 将 USER_ID 参数设置为 1000，可以在 Dockerfile 中使用 ARG USER_ID 来引用这个值。
5. `-m` 2g
   -m 或 --memory: 限制构建过程中容器可以使用的内存量。
   2g: 限制为 2GB 内存。
6. `--network` =host
   --network: 指定构建过程中容器使用的网络模式。
   host: 使用主机的网络栈，这样容器可以直接访问主机的网络资源。
7. `.`
   .: 指定构建上下文的路径，即 Dockerfile 所在的目录。. 表示当前目录。

4.2 镜像命令

- 镜像相关命令
  1. 查看本地镜像

```
docker images
```

  2. 搜索镜像仓库

```
docker search <名称关键字>
```

3. 下载镜像

```
docker pull <镜像名:tag版本>
```

4. 上传镜像

```
docker push <镜像名:tag版本>
```

5. 删除镜像

```
docker rmi <镜像名:tag版本>
```

6. **打包本地镜像文件(单个+多个)**

```
docker save -o <输出文件路径> <镜像名:tag版本>
docker save -o images.tar image_name1:tag1 image_name2:tag2 ...
```

7. 导入本地镜像文件

```
docker load -i <加载文件路径>
docker load -i images.tar
```

4.3 容器命令

- 容器相关的命令

1. 查看本地容器

```
docker ps
```

2. 查看所有容器（包括未运行）

```
docker ps -a
```

3. 启动容器

```
docker start <容器ID>
```

4. 停止容器

```
docker stop <容器ID>
```

5. 删除容器

```
docker rm <容器ID>
```

6. 进入容器内部

```
docker exec -it <容器ID> /bin/bash
docker exec -it <容器ID> /bin/sh
```

7. **容器创建成镜像**

```
docker commit -a "作者信息" -m "log信息" <容器ID> <目标镜像名称: tag版本>
```

8. 拷贝文件到容器

```
docker cp <文件目录> <容器ID>:<目标目录>
```

9. 拷贝容器文件到宿主机

```
docker cp <容器ID>:<文件目录> <宿主机目标目录>
```

10. 更新容器设置

```
docker update <容器ID> <相关设置>
```

- 创建容器命令- 基础参数

```
docker run \

-dit \
--name <容器名称> \

-v /home/worksapce:/home/workspace \

-p 8001:8080 \

-e TZ="Asia/Shanghai" \
-e LD_LIBRARY_PATH="/usr/local/lib" \

ubuntu:22.04 \

bin/bash
```

基础参数

常用参数

-d	容器后台运行
-i	交互模式运行容器
-t	重新分配一个伪输入终端
-dit	-d -i -t省略写法
--name <容器名称>	设置容器名称
--privileged	允许全部root权限
--cpuset="0,1,2"	指定cpu运行
-m 300M	内存限制
docker run --help	查看全部参数

- 创建容器命令- 挂载宿主机目录

```
docker run \

-d \
--name <容器名称> \

-v /home/worksapce:/home/workspace \

-p 8001:8080 \

-e TZ="Asia/Shanghai" \
-e LD_LIBRARY_PATH="/usr/local/lib" \

ubuntu:22.04 \

bin/bash
```

挂载宿主机目录

-v /home/worksapce:/home/workspace

宿主机目录　　容器内目录

容器可直接操作宿主机此目录下的文件

- 创建容器命令- 网络设置

```
docker run \

-dit \
--name <容器名称> \

-v /home/worksapce:/home/workspace \

-p 8001:8080 \                    网络设置

-e TZ="Asia/Shanghai" \
-e LD_LIBRARY_PATH="/usr/local/lib" \

ubuntu:22.04 \

bin/bash
```

设置说明

-p <宿主机端口:容器端口>	端口映射
-h <容器hostname>	指定hostname
--ip 192.168.0.10	指定容器IP
--network=host	共用宿主机网络 其他网络设置将不生效

- 创建容器命令- 环境变量

```
docker run \

-dit \
--name <容器名称> \

-v /home/worksapce:/home/workspace \

-p 8001:8080 \
                                  环境变量
-e TZ="Asia/Shanghai" \
-e LD_LIBRARY_PATH="/usr/local/lib" \

ubuntu:22.04 \

bin/bash
```

设置说明

-e TZ="Asia/Shanghai"　　设置时区

常规Linux设置环境变量方法是不生效的

- 创建容器命令- 容器每次启动执行的命令

```
docker run \

-dit \
--name <容器名称> \

-v /home/worksapce:/home/workspace \

-p 8001:8080 \

-e TZ="Asia/Shanghai" \
-e LD_LIBRARY_PATH="/usr/local/lib" \

ubuntu:22.04 \

bin/bash        容器每次启动执行的命令
```

基础参数中含有-it才会生效

一般为Shell指令
执行完容器会退出

bin/bash可阻止容器退出

4.4 查看日志

- **查看容器的所有日志**（常用）

```
docker logs <container_id_or_name>
```

- **实时查看容器的日志**(类似 tail -f ,常用)

```
docker logs -f <container_id_or_name>
```

- 查看容器的最后几行日志

```
docker logs --tail <number> <container_id_or_name>
```

- 查看从某个时间点开始的日志

```
docker logs --since <timestamp> <container_id_or_name>
```

- 查看到某个时间点为止的日志

```
docker logs --until <timestamp> <container_id_or_name>
```

- 查看日志并显示时间戳

```
docker logs -t <container_id_or_name>
```

4.5 docker推送

- 推送新生成的 docker 镜像到 dockerhub 的步骤

```
# 1. 登录到 Docker Hub
docker login

# 2. 运行镜像
docker run -d --name my_container my_image:1.0

# 3. 为镜像打标签
docker tag image_id:1.0 my_repo/my_image:1.0

# 4. 提交容器（从容器创建新镜像--可选）
# 假设你在容器中进行了一些修改，并希望保存为新的镜像
docker commit container_id my_repo/my_image:1.1

# 5. 推送镜像到远程仓库
docker push my_repo/my_image:1.1
```